

# Informatika 3

Prednáška 10: Funkcie vyššieho rádu

Michal Mrena



ŽILINSKÁ UNIVERZITA V ŽILINE  
Fakulta riadenia  
a informatiky

# Šablóny funkcií



# Príklad

Chceme implementovať funkciu `count`, ktorá vráti počet výskytov vybraného celého čísla (`int`) vo vektore celých čísel (`std::vector<int>`).

```
int count(const std::vector<int> &data, const int element) {  
    int counter = 0;  
    for (const int x : data) {  
        if (x == element) {  
            ++counter;  
        }  
    }  
    return counter;  
}
```

Implementácia funkcie je veľmi jednoduchá. Rovnako ako jej použitie:

```
int main() {  
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};  
    std::cout << count(xs, 2) << "\n";  
}
```

# Príklad

Ak by sme potrebovali rovnakú funkcionálnosť pre vektor reťazcov (`std::string`), museli by sme pridať nové *preťaženie* funkcie:

```
int count(const std::vector<std::string> &data, const std::string &element);
```

Definícia funkcie by bola totožná s definíciou pre typ `int`. Rozdiel by bol len v hlavičke funkcie. Podobných preťažení by sme mohli implementovať niekoľko:

```
int count(const std::vector<double> &data, const double element);  
int count(const std::vector<std::chrono::seconds> &data, const std::chrono::seconds element);  
// ...
```

Takýto postup má všetky nevýhody všeobecne platné pre duplicitný kód.

# Šablóna funkcie

Riešením tohto problému je použitie **šablóny funkcie**. Šablóna bude parametrizovaná jedným typovým parametrom `T` reprezentujúcim typ údajov.

```
template<typename T>
int count(const std::vector<T> &data, const T &element) {
    int counter = 0;
    for (const T &x : data) {
        if (x == element) {
            ++counter;
        }
    }
    return counter;
}
```

# Šablóna funkcie

Riešením tohto problému je použitie **šablóny funkcie**. Šablóna bude parametrizovaná jedným typovým parametrom `T` reprezentujúcim typ údajov.

```
template<typename T>
int count(const std::vector<T> &data, const T &element) {
    int counter = 0;
    for (const T &x : data) {
        if (x == element) {
            ++counter;
        }
    }
    return counter;
}
```

Vďaka dedukcii typových parametrov je použitie funkcie bez zmeny:

```
int main() {
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};
    std::cout << count(xs, 2) << "\n";
}
```

# Šablóna členskej funkcie

Šablónu členskej funkcie deklarujeme rovnakým spôsobom, ako šablónu nečlenskej (voľnej) funkcie:

```
class Chest {  
public:  
    template<typename Hero>  
    void giveLootTo(Hero &hero);  
  
private:  
    Item *m_item;  
};
```

Definícia šablóny členskej funkcie:

```
template<typename Hero>  
void Chest::giveLootTo(Hero &hero) {  
    if (m_item) {  
        hero.receiveLoot(*m_item);  
        m_item = nullptr;  
    }  
}
```

# Šablóna členskej funkcie

Ak je vlastník členskej funkcie sám šablónou, je v (out-of-line) definícii potrebné uviesť šablónu dvakrát – raz pre vlastníka a raz pre člena:

```
template<typename Item>
class Chest {
public:
    template<typename Hero>
    void giveLootTo(Hero &hero);

private:
    std::optional<Item> m_item;
};
```

```
template<typename Item>
template<typename Hero>
void Chest<Item>::giveLootTo(Hero &hero) {
    if (m_item.has_value()) {
        hero.receiveLoot(*m_item);
        m_item.reset();
    }
}
```

# Kontajnery



# Kontajnery

`std::vector` je pravdepodobne najpoužívanější kontajner. Zďaleka však nie je jediný. Ak by sme mali údaje uložené napríklad v kontajneri `std::forward_list`:

```
std::forward_list<std::string> ys{"Marty", "Melman", "Gloria", "Alex",  
    "Donkey", "Shrek", "Fiona", "Donkey"};
```

Potrebovali by sme novú definíciu šablóny funkcie `count` pre tento kontajner:

```
template<typename T>  
int count(const std::forward_list<T> &data, const T &element) {  
    int counter = 0;  
    for (const T &x : data) {  
        if (x == element) {  
            ++counter;  
        }  
    }  
    return counter;  
}
```

# Kontajnery

Riešením tohto problému je parametrizovanie funkcie ďalším typovým parametrom `Container` reprezentujúcim kontajner:

```
template<typename Container, typename T>
int count(const Container &data, const T &element) {
    int counter = 0;
    for (const T &x : data) {
        if (x == element) {
            ++counter;
        }
    }
    return counter;
}
```

# Dedukcia typov

Toto riešenie však vytvorilo nový problém súvisiaci s dedukciou typu `T`, ktorý sa prejaví napríklad pri práci s reťazcami:

```
template<typename Container, typename T>
int count(const Container &data, const T &element) {
    int counter = 0;
    for (const T &x : data) {
// ...
    }
```

```
int main() {
    std::forward_list<std::string> ys{"Marty", "Melman", "Gloria", "Alex", "Donkey",
    "Shrek", "Fiona", "Donkey"};
    std::cout << count(ys, "Donkey") << "\n";
}
```

```
Container = std::forward_list<std::string>
T = const char[7]
```

# Dedukcia typov

Toto riešenie však vytvorilo nový problém súvisiaci s dedukciou typu `T`, ktorý sa prejaví napríklad pri práci s reťazcami:

```
template<>
int count(const std::forward_list<std::string> &data, const char (&element)[7]) {
    int counter = 0;
    for (const char (&x)[7] : data) {
// ...
    }
```

```
int main() {
    std::forward_list<std::string> ys{"Marty", "Melman", "Gloria", "Alex", "Donkey",
    "Shrek", "Fiona", "Donkey"};
    std::cout << count(ys, "Donkey") << "\n";
}
```

```
Container = std::forward_list<std::string>
T = const char[7]
```

Typ `const char (&)[7]` predstavuje odkaz na pole typu `char` o veľkosti 7.

# Dedukcia typov

Toto riešenie však vytvorilo nový problém súvisiaci s dedukciou typu `T`, ktorý sa prejaví napríklad pri práci s reťazcami:

```
template<>
int count(const std::forward_list<std::string> &data, const char (&element)[7]) {
    int counter = 0;
    for (const char (&x)[7] : data) {
// ...
    }
```

```
error: invalid initialization of reference of type 'const char (&)[7]' from expression of
type 'const std::__cxx11::basic_string<char>'
 29 |     for (const T &x : data) {
    |           ^~~
```

# Dedukcia typov

Riešením tohto problému je zadať typ parametra `element` tak, aby bol rovnaký ako typ prvkov uložených v kontajneri `Container`.

Všetky kontajnery zo štandardnej knižnice obsahujú verejný členský alias typu `value_type`, ktorý zodpovedá typu uchovávaných prvkov a niekoľko ďalších aliasov:

```
template<typename T>
class forward_list {
public:
    using value_type = T;
    using reference = T&;
    using const_reference = const T&;
    using iterator = forward_list_iterator;
    // ...
};
```

# Závislý typ

Upravená definícia funkcie `count` tak môže vyzeráť nasledovne:

```
template<typename Container>
int count(const Container &data, const typename Container::value_type &element) {
    int counter = 0;
    for (const auto &x : data) {
        if (x == element) {
            ++counter;
        }
    }
    return counter;
}
```

Typ `value_type` je v tomto prípade závislý na typovom parametri `Container`. Pri použití je preto potrebné použiť kľúčové slovo `typename`.

Tento typ by sme mohli použiť aj pri definícii premennej `x`. V tomto prípade je však jednoduchšie použiť kľúčové slovo `auto`, ktoré zabezpečí, že kompilátor určí typ premennej automaticky.

# Iterátory



# Príklad

Aktuálna definícia funkcie `count` je už pomerne generická – môžeme ju použiť s ľubovoľným kontajnerom spĺňajúcim niekoľko predpísaných vlastností.

```
template<typename Container>
int count(const Container &data, const typename Container::value_type &element);
```

**Príklad:** Početnosť výskytu prvku chceme hľadať iba v druhej polovici kontajnera. V aktuálnej implementácii by sme museli vytvoriť kópiu polovice kontajnera:

```
int main() {
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};
    std::vector<int> half(xs.begin(), xs.begin() + xs.size() / 2);
    std::cout << count(half, 2) << "\n";
}
```

(Od C++20) by sme mohli použiť [pohľad](#) `std::span`, čím by sme sa vyhli alokovaniu pamäte. \*

# Iterátory

Všetky kontajnery zo štandardnej knižnice poskytujú iterátory. Preto je lepšie algoritmy parametrizovať iterátormi namiesto kontajnerov:

```
template<typename InputIt, typename T>
int count(InputIt first, InputIt last, const T &element) {
    int counter = 0;
    for (; first != last; ++first) {
        if (*first == element) {
            ++counter;
        }
    }
    return counter;
}
```

```
int main() {
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};
    std::cout << count(xs.begin(), xs.end(), 2) << "\n";
}
```

Iterátor vektora môžeme posúvať o niekoľko prvkov dopredu **operátorom** `+`. \*

# Iterátory

Algoritmus parametrizovaný iterátorom môžeme použiť s ľubovoľným kontajnerom alebo časťou kontajnera, bez nutnosti vytvárania dočasných *pomocných* kontajnerov.

```
template<typename InputIt, typename T>
int count(InputIt first, InputIt last, const T &element);

int main() {
    std::forward_list<std::string> ys{"Marty", "Melman", "Gloria", "Alex", "Donkey", "Shrek",
    "Fiona", "Donkey"};
    std::forward_list<std::string>::iterator first = ys.begin();
    std::forward_list<std::string>::iterator last = ys.end();
    std::advance(first, 2);
    std::cout << count(first, last, "Donkey") << "\n";
}
```

Iterátor kontajnera `std::forward_list` nemôžeme posunúť **operátorom** `+`. Môžeme na to však použiť funkciu `std::advance` z hlavičky `<iterator>`.

# Kategórie iterátorov

- Každý iterátor definuje preťaženia operátorov s uvedenou sémantikou:
  - `operator++` (prefixový) – posun na ďalší prvok
  - `operator*` – sprístupnenie aktuálneho prvku
  - `operator!=` – porovnanie s iným iterátorom (z rovnakého kontajnera)
- Definuje tiež niekoľko (okrem iných) verejných členských aliasov typov
- Podľa ďalších podporovaných operácií (preťažených operátorov) delíme iterátory do **kategórií**
- Rôzne algoritmy na svoje fungovanie vyžadujú rôzne kategórie iterátorov
- Štandardná knižnica obsahuje mnoho algoritmov naprogramovaných voči iterátorom v knižnici `<algorithm>`

# Vstupný iterátor

- Vstupný iterátor je najjednoduchší typ iterátora
- Ide o tzv. **single-pass** iterátor – garantuje *jeden* prechod kontajnerom (vo všeobecnosti vstupom), ktorý sprístupňuje
- Nie je možné sprístupniť už navštívené prvky
- Nie je možné zapisovať cez *odkaz* vrátený cez **operator\***

```
template<typename InputIt, typename T>
int count(
    InputIt first,
    InputIt last,
    const T &element
) {
    int counter = 0;
    for (; first != last; ++first) {
        if (*first == element) {
            ++counter;
        }
    }
    return counter;
}
```

# Výstupný iterátor

- Výstupný iterátor je špeciálny typ iterátora slúžiaci na zápis/vkladanie prvkov
- Ide o **single-pass** iterátor – garantuje iba jeden zápis v aktuálnej pozícii
- Je možné iba zapisovať cez *odkaz* vrátený cez **operator\***
- Používa sa na implementáciu univerzálneho výstupu algoritmov

```
template<typename InputIt, typename OutputIt>
void select_primes(
    InputIt first,
    InputIt last,
    OutputIt out
) {
    for (; first < last; ++first) {
        const auto num = *first;
        if (is_prime(num)) {
            *out++ = num;
        }
    }
}
```

# Dopredný iterátor

- Dopredný iterátor je vstupný iterátor
- Podporuje opakovanú prehliadku kontajnera
- Ide o **multi-pass** iterátor
- Môže vystupovať ako vstupný aj výstupný iterátor
- Môže podporovať modifikáciu aktuálneho prvku

```
template<typename ForwardIt>
bool contains_duplicate(
    ForwardIt first,
    ForwardIt last
) {
    for (ForwardIt i = first; i != last; ++i) {
        for (ForwardIt k = i; k != last; ++k) {
            if (i != k && *i == *k) {
                return true;
            }
        }
    }
    return false;
}
```

# Obojsmerný iterátor

- Obojsmerný iterátor je dopredný iterátor
- Umožňuje posunúť iterátor v opačnom smere na už sprístupnené prvky
- Na posun v opačnom smere používa **operator--**

```
template<typename BidirIt>
bool is_palindrome(BidirIt first, BidirIt last) {
    --last;
    for (;first != last;) {
        if (*first != *last) {
            return false;
        }
        ++first;
        if (first != last) {
            --last;
        }
    }
    return true;
}
```

# Iterátor s náhodným prístupom

- Iterátor s náhodným prístupom má bohaté rozhranie
- Môžeme s ním vykonávať takmer všetky operácie, ktoré bežne vykonávame s indexom
- Všetky operácie vykonáva **efektívne** – napríklad operácie posunu o niekoľko prvkov dopredu je možné implementovať aj na *doprednom iterátore* – nie však *efektívne*

```
template<typename RandomIt, typename T>
bool contains_sorted(
    RandomIt first,
    RandomIt last,
    const T &element
) {
    while (first < last) {
        RandomIt mid = first + (last - first) / 2;
        if (*mid == element) {
            return true;
        } else if (*mid < element) {
            first = mid + 1;
        } else {
            last = mid;
        }
    }
    return false;
}
```

# Funkcie vyššieho rádu



# Príklad

Chceme implementovať funkciu `count_if`, ktorá vráti počet prvkov, ktoré majú nejakú vlastnosť. Aby bola funkcia všeobecná, musí kontrolu vlastnosti poskytnúť jej používateľ:

```
template<typename InputIt>
int count_if(InputIt first, InputIt last, ??? check) {
    int counter = 0;
    for (; first != last; ++first) {
        if (check(*first)) {
            ++counter;
        }
    }
    return counter;
}
```

Implementácia si vyžaduje pracovať s funkciou ako s hodnotou. V jazyku C++ to môžeme dosiahnuť niekoľkými spôsobmi.

# Ukazovateľ na funkciu

- Identifikátor funkcie zároveň slúži aj ako ukazovateľ na danú funkciu
- Typ takéhoto ukazovateľa je daný hlavičkou funkcie a zapisujeme ho špeciálnou syntaxou:

<návratový typ>(\*)(<typy parametrov>)

- V jazyku C++ sa nepoužíva často

```
int add(int a, int b) {  
    return a + b;  
}  
  
int processInts(int a, int b, int(*)(int,int)) {  
    return f(a,b);  
}  
  
int main() {  
    int(*operation)(int,int) = add;  
    const int result = processInts(  
        10,  
        20,  
        operation);  
    std::cout << result << „\n“;  
}
```

# Ukazovateľ na funkciu

Ak chceme ukazovateľ na funkciu použiť v implementácii funkcie `count_if`, potrebujeme získať typ sprístupňovaný iterátorom `InputIt`. To je možné, avšak syntakticky náročné:

```
template<typename InputIt>
int count_if(
    InputIt first,
    InputIt last,
    bool (*check)(typename std::iterator_traits<InputIt>::reference)
) {
    int counter = 0;
    for (;first != last; ++first) {
        if (check(*first)) {
            ++counter;
        }
    }
    return counter;
}
```

# Funkčný objekt

- Jazyk C++ umožňuje preťažiť `operator()`
- K objektu (inštancii) tak môžeme pristupovať ako ku funkcii so zachovaním benefitov (napríklad možnosť uchovávať stav)
- Objekt preťažujúci `operator()` nazývame funkčný objekt\*
- Funkčné objekty sú často veľmi jednoduché – okrem preťaženého operátora toho veľa neobsahujú

```
struct Adder {  
    int operator()(int a, int b) {  
        ++additionsPerformed;  
        return a + b;  
    }  
  
    int additionsPerformed{0};  
};  
  
int main() {  
    Adder adder;  
    const int result = adder(1,2) + adder(30,10);  
    std::cout << result << "\n";  
    std::cout << adder.additionsPerformed << "\n";  
}
```

Často nesprávne označovaný ako Functor. \*

# Funkčný objekt

- Štandardná knižnica poskytuje niekoľko funkčných objektov v hlavičke `<functional>`
- Zväčša implementuje aritmetické a relačné operácie
- Veľmi dobre spolupracujú s knižnicou `<algorithm>`
- Funkčné objekty často používame ako anonymné objekty
- Dobre fungujú s dedukciou typov

```
int main() {  
    std::plus<int> adder;  
    std::multiplies<int> multiplier;  
    const int result =  
        adder(1,2) + multiplier(30,10);  
    std::cout << result << "\n";  
}
```

```
int main() {  
    const int result =  
        std::plus<>()(1,2) +  
        std::multiplies<>()(30,10);  
    std::cout << result << "\n";  
}
```

Často nesprávne označovaný ako Functor. \*

# Funkčný objekt

Ak chceme umožniť použitie funkcie `count_if` s rôznymi funkčnými objektami alebo ukazovateľom na funkciu, musíme ju parametrizovať typom funkcie kontrolujúcej vlastnosť prvkov:

```
template<typename InputIt, typename Predicate>
int count_if(InputIt first, InputIt last, Predicate check) {
    int counter = 0;
    for (; first != last; ++first) {
        if (check(*first)) {
            ++counter;
        }
    }
    return counter;
}
```

Funkciu, ktorá preberá objekt a vráti hodnotu typu `bool` podľa toho, či objekt má nejakú vlastnosť, často označujeme ako *predikát*.

# Funkčný objekt

Po pridaní typového parametra tak ako *predikát* môžeme použiť inštanciu ľubovoľného typu, ktorý definuje `operator()`.

```
template<typename InputIt, typename Predicate>
int count_if(InputIt first, InputIt last, Predicate check);

bool is_even(int x) {
    return !(x & 1);
}

struct IsOdd {
    bool operator()(int x) const {
        return x & 1;
    }
};

int main() {
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};
    std::cout << count_if(xs.begin(), xs.end(), is_even) << "\n";
    std::cout << count_if(xs.begin(), xs.end(), IsOdd()) << "\n";
}
```

# Funkcie vyššieho rádu

- Funkcia, ktorá:
  - preberá inú funkciu ako parameter
  - alebo vracia novú funkciu
- sa označuje ako funkcia vyššieho rádu
- Všetky funkcie, s ktorými sme pracovali doteraz boli funkcie prvého rádu
- Takéto funkcie sú esenciálnou súčasťou funkcionálneho programovania
- V imperatívnom programovaní sa najčastejšie vyskytujú v generickom kóde – napríklad v knižnici `<algorithm>`

# Lambda funkcie



# Lambda funkcie

- Funkčné objekty alebo ukazovatele na funkcie často použijeme práve jedenkrát v mieste volania funkcie vyššieho rádu
- Vytvárať nový typ kvôli jednému použitiu je *neefektívne*\*
- Väčšina imperatívnych jazykov<sup>^</sup> preto v nejakej forme obsahuje lambda funkcie
- Lambda funkcia je **anonymná funkcia** definovaná výrazom

```
import Data.List

main :: IO ()
main = putStr
    $ show
    $ sortBy (\l r -> compare (length l) (length r))
      ["Marty", "Melman", "Alex", "Shrek", "Fiona"]
```

Lambda funkcia v jazyku Haskell

```
names = sorted(
    ["Marty", "Melman", "Alex", "Shrek", "Fiona"],
    key=lambda x: len(x))
print(names)
```

Lambda funkcia v jazyku Python

Nie z pohľadu výkonu, ale z pohľadu práce a množstva kódu. \*

Vo funkcionálnych jazykoch sú esenciálnou súčasťou jazyka. ^





Alan Turing



Alonzo Church



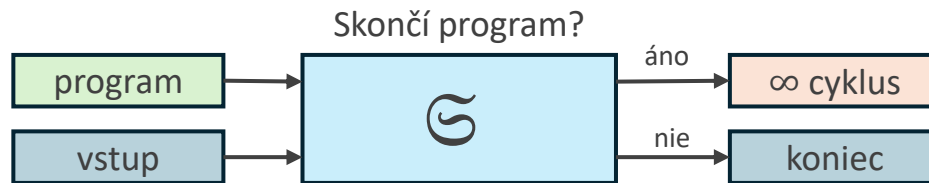
Gordon Freeman

# Lambda funkcie

- Lambda funkcie pochádzajú z tridsiatich rokov 20. storočia
- Sú súčasťou matematického nástroja lambda kalkulus
- Alonzo Church pomocou neho vyriešil Entscheidungsproblem:  
„Existuje univerzálny algoritmus, ktorý pre každé tvrdenie v logike prvého rádu rozhodne (v konečnom čase) či je tvrdenie platné?“
- Alan Turing ho nezávisle vyriešil pomocou Turingovho stroja
- Obaja potrebovali matematický model výpočtov
  - Nimi navrhnuté modely výpočtov sú základom pre súčasné programovacie jazyky

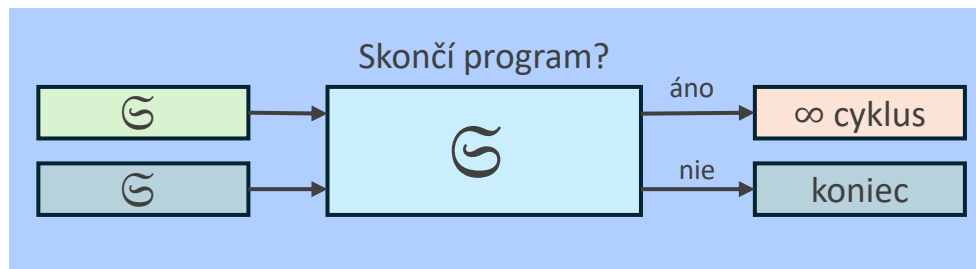
# Entscheidungsproblem

- Alan Turing vyriešil E riešením podobného problému známy ako [Halting Problem](#)  
„Existuje univerzálny program, ktorý pre akýkoľvek program a jeho vstup povie, či program skončí?“
- [Turingov dôkaz](#) je dôkaz sporom – predpokladajme, že takýto program existuje, označme ho  $\mathcal{G}$



# Entscheidungsproblem

- Alan Turing vyriešil E riešením podobného problému známy ako [Halting Problem](#)  
„Existuje univerzálny program, ktorý pre akýkoľvek program a jeho vstup povie, či program skončí?“
- [Turingov dôkaz](#) je dôkaz sporom – predpokladajme, že takýto program existuje, označme ho  $\mathcal{G}$
- Ak vložíme  $\mathcal{G}$  ako vstupný program aj samotný vstup, dostávame paradox – predpoklad neplatí



# Lambda funkcie v jazyku C++

- V jazyku C++ má lambda funkcia nasledovnú syntax:

```
[<captures>](<parametre>) -> <ret> {  
    <telo funkcie>  
}
```

- Kde:
  - <parameter> – zoznam parametrov
  - <ret> – návratový typ (nepovinný, kompilátor ho môže vydedukovať)
  - <captures> – zoznam zachytených premenných, zachytené môže byť odkazom alebo hodnotou

```
int main() {  
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};  
    const int num = 2;  
    int comparisonCount = 0;  
    const int result = count_if(xs.begin(), xs.end(),  
        [num, &comparisonCount](const int x) -> bool {  
            ++comparisonCount;  
            return x == num;  
        });  
}
```

# Lambda funkcie v jazyku C++

Lambda funkciu môžeme uložiť do premennej s použitím kľúčového slova `auto`:

```
int main() {  
    std::vector<int> xs{1,2,2,2,3,4,5,6,1};  
    const int num = 2;  
    int comparisonCount = 0;  
    auto f = [num, &comparisonCount](const int x) {  
        ++comparisonCount;  
        return x == num;  
    };  
    const int result = count_if(xs.begin(), xs.end(), f);  
}
```

Kompilátor automaticky vytvorí funkčný objekt uchovávajúci zachytené premenné.

```
class __lambda_7_14 {  
public:  
    bool operator()(const int x) const {  
        ++comparisonCount;  
        return x == num;  
    }  
  
private:  
    const int num;  
    int &comparisonCount;  
  
public:  
    __lambda_7_14(  
        const int &_num,  
        int &_comparisonCount  
    ) :  
        num{_num},  
        comparisonCount{_comparisonCount}  
    {  
    }  
};
```

# Lambda funkcie v jazyku C++

Definícia lambda funkcie je *výraz* – na rozdiel od definícií funkcií a funkčných objektov, kde ide o *príkazy*.

Preto s ňou môžeme pracovať ako s akoukoľvek inou hodnotou:

```
// Immediately invoked lambda expression.
std::cout << [](){ return 5*7; }() << "\n";

// Lambda funkcia, ktorá vytvára lambda funkciu.
auto create_constant = [](const auto x){
    return [x](auto const&){
        return x;
    };
};
auto c7 = create_constant(7);
std::cout << c7(std::tuple<>()) << "\n";
```

Lambda funkcie tak ponúkajú zaujímavé možnosti pre funkcionálne programovanie v C++.

# Štandardná knižnica



# Štandardná knižnica

- Štandardná knižnica obsahuje pomerne širokú podporu (generického) funkcionálneho programovania:
  - Knižnica `<algorithm>`
  - Knižnica `<functional>`
  - Knižnica `<numeric>`

# Univerzálny funkčný objekt

- V niektorých situáciách nie je možné parametrizovať kód šablónou:
  - Ak potrebujeme funkčný objekt (resp. čokoľvek, čo je možné zavolať **operátorom()** – callable) uchovávať v premennej
  - Ak potrebujeme parameter typu *callable* objekt vo virtuálnej funkcii
- (Od C++11) V takýchto situáciách môžeme použiť (meta) funkčný objekt `std::function`
- Pri svojom vytvorení si uloží iný *callable* objekt
- Vo svojej implementácii operátora() preposiela parametre spravovaného *callable* objektu
- Na implementáciu využíva techniku type erasure
- Oproti použitiu šablóny tak pridáva jednu vrstvu, cez ktorú kompilátor nevidí – podobne, ako pri virtuálnych funkciách

# Univerzálny funkčný objekt

Často sa používa na implementáciu callback a hook funkcií.

Typ `void(const SimulationResult&)` je typom funkcie.

V tomto prípade funkcie s návratovým typom `void` a jedným parametrom typu

`const SimulationResult&`.

```
class Simulation {
public:
    Simulation() :
        m_afterSimulation(
            [] (const SimulationResult&) { /* do nothing */ }
        )
    {}
    template<typename F>
    void registerAfterSimulation(F f) {
        m_afterSimulation = std::move(f);
    }
    void simulate() {
        SimulationResult result;
        // ...
        m_afterSimulation(result);
    }
private:
    std::function<void(const SimulationResult&)> m_afterSimulation;
};
```

# Čiastočná aplikácia funkcií

- Vo funkcionálnom programovaní je dôležitá technika čiastočnej aplikácie funkcií (angl. partial function application)
- Zafixovaním  $k$  parametrov funkcie s  $n$  parametrami vznikne nová funkcia s  $n - k$  parametrami
- (Od C++20) môžeme na vytvorenie nového funkčného objektu z existujúceho zafixovaním prvých parametrov použiť funkciu `std::bind_front`

```
bool compare(int a, int b) {  
    return a < b;  
}  
  
int main() {  
    auto is_greater_than_7 = std::bind_front(compare, 7);  
    std::cout << is_greater_than_7(20) << "\n";  
}
```

Existujú tiež funkcie `std::bind_back` a `std::bind`. Rovnaký výsledok môžeme docieľiť použitím lambda funkcie.

# Práca s funkčnými objektami

- Okrem *callable* objektov doposiaľ uvedených v prednáške existuje niekoľko ďalších vrátane ukazovateľov na členov, zástupných objektov, ...
- Pri takýchto typoch objektov nefunguje priame volanie `operátora()`
- V generickom kóde je preto pri volaní `operátora()` nad neznámym typom (reprezentovaným typovým parametrom) lepšie použiť funkciu `std::invoke`, ktorá korektne ošetruje aj rôzne okrajové prípady

```
template<typename InputIt, typename Predicate>
int count_if(InputIt first, InputIt last, Predicate check) {
    int counter = 0;
    for (; first != last; ++first) {
        if (std::invoke(check, *first)) {
            ++counter;
        }
    }
    return counter;
}
```

# Pokročilé šablóny



# Preposielaný odkaz

- Trieda `Chest` uchováva inštanciu parametrizovaného typu `Item`
- Členskú premennú môžeme inicializovať *kopírovacím* alebo *move* konštruktorom
- (10) inicializuje člena `m_item` jeho *move* konštruktorom
- Výraz `item` typu `Item&&` je **lvalue**, preto je nutné pretypovanie s použitím `std::move`

```
1  template<typename Item>
2  class Chest {
3  public:
4      explicit Chest(const Item &item) :
5          m_item(item)
6      {
7      }
8
9      explicit Chest(Item &&item) noexcept :
10         m_item(std::move(item))
11     {
12     }
13
14 private:
15     Item m_item;
16 };
```

# Preposielaný odkaz

- V generickom kóde často nájdeme dve preťaženia funkcie pre lvalue a rvalue odkaz
- V niektorých situáciách je možné nahradiť ich jednou šablónou funkcie, ktorá preberá tzv. preposielaný odkaz (angl. forwarding reference)
- Pri použití preposielaného odkazu je zachovaná kategória hodnoty

```
1 template<typename Item>
2 class Chest {
3 public:
4     template<typename I>
5     explicit Chest(I &&item) :
6         m_item(std::forward<I>(item))
7     {
8     }
9
10 private:
11     Item m_item;
12 };
```

# Preposielaný odkaz

```
1 struct GlovesOfHaste {
2 };
3
4 int main() {
5     GlovesOfHaste gloves;
6     Chest<GlovesOfHaste> chest1(gloves);
7     Chest<GlovesOfHaste> chest2(GlovesOfHaste{});
8 }
```

(6) `I = GlovesOfHaste&` typ parametra je tak `GlovesOfHaste& &&item`.

Pri odkazoch funguje spájanie odkazov. Po spojení bude typ parametra lvalue odkaz `GlovesOfHaste &item`.

(7) `I = GlovesOfHaste` typ parametra je tak `GlovesOfHaste &&item`.

Bez potreby spájania odkazov je typ parametra rvalue odkaz.

```
1 template<typename Item>
2 class Chest {
3 public:
4     template<typename I>
5     explicit Chest(I &&item) :
6         m_item(std::forward<I>(item))
7     {
8     }
9
10 private:
11     Item m_item;
12 };
```

# Preposielaný odkaz

```
1 struct GlovesOfHaste {  
2 };  
3  
4 int main() {  
5     GlovesOfHaste gloves;  
6     Chest<GlovesOfHaste> chest1(gloves);  
7     Chest<GlovesOfHaste> chest2(GlovesOfHaste{});  
8 }
```

```
1 template<typename Item>  
2 class Chest {  
3 public:  
4     template<typename I>  
5     explicit Chest(I &&item) :  
6         m_item(std::forward<I>(item))  
7     {  
8     }  
9  
10 private:  
11     Item m_item;  
12 };
```

Funkcia `std::forward` je podobná funkcii `std::move`.  
Jej úlohou je pretypovať parameter tak, aby zostala zachovaná jeho kategória hodnoty.

# Balík typových parametrov

- V príklade sme doposiaľ inicializovali členskú premennú podľa stavu inej inštancie rovnakého typu (ktorá musela za týmto účelom vzniknúť)
- Efektívnejší prístup je vytvoriť inštanciu (členskú premennú) priamo na mieste
- Vo všeobecnosti však nepoznáme počet jej parametrov
- Riešením je použitie balíka typových parametrov

```
template<typename Item>
class Chest {
public:
    template<typename... Args>
    Chest(Args &&...item) :
        m_item(std::forward<Args>(item)...)
    {
    }

private:
    Item m_item;
};
```

# Balík typových parametrov

```
struct GlovesOfHaste {  
    GlovesOfHaste(  
        int durability,  
        double multiplier  
    );  
};  
  
int main() {  
    Chest<GlovesOfHaste> chest(10, 1.15);  
}
```

Inštancia triedy `GlovesOfHaste` je inicializovaná priamo v inštancii triedy `Chest<GlovesOfHaste>` jej parametrickým konštruktorom so zachovaním kategórie hodnôt parametrov.

```
template<typename Item>  
class Chest {  
public:  
    template<typename... Args>  
    Chest(Args &&...item) :  
        m_item(std::forward<Args>(item)...)  
    {  
    }  
  
private:  
    Item m_item;  
};
```

# Zhrnutie

- Šablóny funkcií v spojení s iterátormi umožňujú tvorbu generických algoritmov
- Tieto algoritmy sú často funkciami vyššieho rádu – na svoje fungovanie potrebujú inú funkciu, ktorú preberú ako parameter
- Ako parameter môžeme použiť napríklad funkčný objekt alebo lambda funkciu
- Pri pokročilom generickom programovaní môžeme použiť nástroje štandardnej knižnice a jazyka – napríklad preposielané odkazy a balíky typových parametrov

# Ďakujem za pozornosť

Priestor na otázky

